Application Sandboxing System - Final Year Project

ESGI Paris - Systems, Networks and Security (2025-2026)

---

1. PROJECT OVERVIEW

1.1 Project Title

Secure Application Sandboxing System in Rust: A Default-Deny Isolation Framework for Untrusted Binaries

1.2 Project Summary

Development of a low-level application sandbox for Linux systems that isolates untrusted binaries using kernel-level security primitives. Unlike container runtimes (Docker, Podman) which focus on application deployment, this tool implements a default-deny security model where applications receive zero privileges by default and must be explicitly granted specific system resources, syscalls, and filesystem access.

1.3 Problem Statement

Current desktop Linux systems grant applications full access to user resources by default. A malicious PDF reader, compromised browser plugin, or untrusted binary can access the entire home directory, make arbitrary network connections, and execute any syscall the user is permitted to run. Existing solutions like Firejail use blacklist-based approaches that can be bypassed, and container technologies like Docker are designed for deployment rather than desktop security.

1.4 Project Objectives

1. Implement a memory-safe sandbox using Rust to eliminate common C-based vulnerabilities
2. Create a default-deny permission model with explicit resource grants
3. Develop an audit mode that automatically discovers application requirements
4. Build a user-friendly profile system for common application types
5. Demonstrate superior isolation compared to existing tools
6. Provide comprehensive security analysis and escape testing

1.5 Key Differentiators

- **Default-deny security model** vs Firejail's blacklist approach
- **Memory-safe implementation** in Rust vs C-based alternatives
- **Explicit permission grants** similar to mobile OS permission systems
- **Per-application sandboxing** for desktop use cases vs container-based deployment
- **Automated profile generation** through syscall learning mode
- **Modern kernel features**: Landlock LSM, cgroups v2, user namespaces

---

2. TECHNICAL ARCHITECTURE

2.1 Core Technologies

Programming Language: Rust

Rationale for Rust:

- Memory safety without garbage collection (eliminates buffer overflows, use-after-free)
- Zero-cost abstractions for syscall interfaces
- Strong type system prevents common security vulnerabilities
- Growing ecosystem for systems programming (nix, libseccomp crates)
- Industry adoption for security-critical systems (Firecracker, youki)

Target Platform: Linux (Ubuntu 22.04+, Arch Linux)

- Kernel version: 5.13+ (required for Landlock LSM)
- Architecture: x86_64 (primary), ARM64 (secondary)

2.2 Linux Kernel Primitives

Namespaces (Process Isolation)

Kernel feature that provides isolated views of system resources:

Mount Namespace (CLONE_NEWNS)

- Purpose: Filesystem isolation
- Implementation: Create isolated mount tree with only whitelisted paths
- Use case: Application sees only /tmp, /usr, and explicitly granted directories

PID Namespace (CLONE_NEWPID)

- Purpose: Process isolation
- Implementation: Application sees isolated process tree (appears as PID 1)
- Use case: Cannot enumerate or signal other user processes

Network Namespace (CLONE_NEWNET)

- Purpose: Network isolation
- Implementation: Isolated network stack, no interfaces by default
- Use case: Complete network denial or restricted network via veth pairs

User Namespace (CLONE_NEWUSER)

- Purpose: UID/GID mapping
- Implementation: Application runs as “root” inside namespace but unprivileged outside
- Use case: Enables rootless sandboxing without setuid binaries

IPC Namespace (CLONE_NEWIPC)

- Purpose: Inter-process communication isolation
- Implementation: Isolated System V IPC objects and POSIX message queues
- Use case: Prevents shared memory attacks

UTS Namespace (CLONE_NEWUTS)

- Purpose: Hostname/domain isolation
- Implementation: Isolated hostname and NIS domain name
- Use case: Prevents hostname fingerprinting

Seccomp-BPF (Syscall Filtering)

Berkeley Packet Filter-based syscall interception:

Default-Deny Model

```
Default action: ERRNO (deny all syscalls)
Whitelist approach: Explicitly allow required syscalls
```

Syscall Groups

- `io` : read, write, open, close, lseek, fstat, stat
- `memory` : mmap, mprotect, munmap, brk, madvise
- `threading` : clone, futex, set_tid_address, set_robust_list
- `network` : socket, connect, send, recv, bind, listen
- `process` : fork, execve, wait4, kill, getpid

Implementation Strategy

- Use libseccomp Rust bindings for BPF filter generation
- Support both allowlist and denylist modes
- Allow conditional rules (e.g., allow open() only for specific paths)

Capabilities (Privilege Separation)

POSIX capabilities for fine-grained privilege control:

Default: Drop ALL capabilities

- CAP_SYS_ADMIN (mount, namespace manipulation)
- CAP_NET_ADMIN (network configuration)
- CAP_SYS_PTRACE (process debugging)
- CAP_SYS_MODULE (kernel module loading)
- CAP_DAC_OVERRIDE (bypass file permissions)
- ... and 35+ other capabilities

Selective Granting

- Most applications require zero capabilities

- Network servers might need CAP_NET_BIND_SERVICE for port 80/443
- Profile can specify: `allow_capabilities = []` (default)

Cgroups v2 (Resource Limits)

Modern unified cgroup hierarchy for resource control:

Memory Controller

- Hard limit: `memory.max = 512M` (kill if exceeded)
- Soft limit: `memory.high = 384M` (throttle if exceeded)
- Swap limit: `memory.swap.max = 0` (disable swap)

CPU Controller

- CPU shares: `cpu.weight = 100` (default 100, max 10000)
- CPU quota: `cpu.max = "50000 100000"` (50% of one core)

I/O Controller

- Read/write IOPS limits
- Bandwidth limits (MB/s)

Process Number Controller (PIDs)

- `pids.max = 50` (limit fork bombs)

Landlock LSM (Modern Access Control)

Kernel 5.13+ feature for filesystem access control:

Advantages over Seccomp for Filesystem

- More expressive than seccomp for file operations
- Allows path-based rules (not just syscall-based)
- Stackable with other LSMs (AppArmor, SELinux)

Access Rights

- `LANDLOCK_ACCESS_FS_READ_FILE` : Read file contents
- `LANDLOCK_ACCESS_FS_WRITE_FILE` : Write to files
- `LANDLOCK_ACCESS_FS_EXECUTE` : Execute files
- `LANDLOCK_ACCESS_FS_READ_DIR` : List directory
- `LANDLOCK_ACCESS_FS_MAKE_REG` : Create regular files
- And more...

Example Rule

```
PathBeneath::new("/tmp", AccessFs::WriteFile | AccessFs::MakeReg)
PathBeneath::new("/usr", AccessFs::ReadFile | AccessFs::Execute)
```

2.3 Rust Crate Dependencies

Core System Interface

```
[dependencies]
# Syscall wrappers (namespaces, mount, fork, exec)
nix = { version = "0.27", features = ["mount", "sched", "process", "user"] }

# Seccomp-BPF filter generation
libseccomp = "0.3"

# POSIX capability manipulation
caps = "0.5"

# Cgroups v2 management
cgroups-rs = "0.3"

# Landlock LSM (optional, requires kernel 5.13+)
landlock = "0.3"
```

CLI and Configuration

```
# Command-line argument parsing
clap = { version = "4.0", features = ["derive"] }

# TOML profile parsing
serde = { version = "1.0", features = ["derive"] }
toml = "0.8"

# User/group information
users = "0.11"
```

Audit Mode (Optional Phase 2)

```
# Process tracing for audit mode
nix = { features = ["ptrace"] }

# Alternatively, eBPF for lower overhead
aya = "0.12"  # eBPF library in pure Rust
```

---

3. SYSTEM ARCHITECTURE

3.1 Component Diagram

```
┌──────────────────────────────────────────────────────────┐
│                    CLI Interface (clap)                  │
│  sandbox-rs [OPTIONS] <BINARY> [ARGS...]                 │
└──────────────────────────┬───────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│                  Profile Engine (TOML)                   │
│  - Parse .toml files or CLI flags                        │
```

```
│  - Resolve syscall groups → syscall numbers                  │
│  - Validate permissions                                      │
│  - Build IsolationConfig struct                              │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│                    Isolation Engine                          │
├──────────────────────────────────────────────────────────────┤
│  1. Namespace Setup                                          │
│     - unshare(CLONE_NEWNS|NEWPID|NEWNET|NEWUSER|...)        │
│  2. Filesystem Builder                                       │
│     - mount tmpfs as new root                                │
│     - bind mount whitelisted paths                           │
│     - pivot_root() to new filesystem                         │
│  3. Network Setup (conditional)                              │
│     - Create veth pair if network allowed                    │
│     - Configure network namespace                            │
│  4. Cgroup Configuration                                     │
│     - Create cgroup                                          │
│     - Set memory, CPU, PID limits                            │
│     - Add process to cgroup                                  │
│  5. Capability Dropping                                      │
│     - Drop all capabilities                                  │
│  6. Seccomp Filter                                           │
│     - Build BPF filter from allowed syscalls                 │
│     - Load filter (irreversible)                             │
│  7. Execute Target                                           │
│     - execve(binary, args, env)                              │
└──────────────────────────────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│             Sandboxed Application Process                    │
│  - Isolated filesystem (only sees whitelisted paths)         │
│  - Isolated PID tree (cannot see other processes)            │
│  - Restricted syscalls (only whitelisted ones work)          │
│  - Network isolated or restricted                            │
│  - Resource limited (memory, CPU, PIDs)                      │
└──────────────────────────────────────────────────────────────┘
```

3.2 Execution Flow

```
┌──────────────────────────────────────────────────────────────┐
│ 1. Parse CLI arguments or load profile                       │
│    Input: --profile firefox.toml firefox                     │
│    Output: IsolationConfig struct                            │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 2. Validate configuration                                    │
│    - Check binary exists and is executable                   │
│    - Verify whitelisted paths exist                          │
│    - Validate syscall names                                  │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
```

```
│ 3. Fork process                                              │
│    Parent: Monitor child, handle signals                     │
│    Child: Continue to step 4                                 │
└──────────────────────────┬──────────────────────────────────┘
                           │ (child process)
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 4. Create namespaces (unshare syscall)                       │
│    CLONE_NEWNS | NEWPID | NEWNET | NEWUSER | ...             │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 5. Setup mount namespace                                     │
│    a. Create tmpfs as new root                               │
│       mount("tmpfs", "/tmp/sandbox_root", "tmpfs", ...)      │
│    b. Bind mount whitelisted paths                           │
│       mount("/usr", "/tmp/sandbox_root/usr", MS_BIND|RO)     │
│       mount("/tmp", "/tmp/sandbox_root/tmp", MS_BIND|RW)     │
│    c. Pivot to new root                                      │
│       pivot_root("/tmp/sandbox_root", ".../old_root")        │
│       umount2("/old_root", MNT_DETACH)                       │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 6. Setup network (if allowed)                                │
│    mode=none: Do nothing (isolated network namespace)        │
│    mode=full: Move process back to host network NS           │
│    mode=restricted: Create veth pair, configure bridge       │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 7. Configure cgroups                                         │
│    - Create /sys/fs/cgroup/sandbox-rs/<pid>                  │
│    - Write memory.max, cpu.max, pids.max                     │
│    - Add PID to cgroup.procs                                 │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 8. Drop capabilities                                         │
│    capset() to remove all capabilities                       │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 9. Apply Landlock rules (if kernel supports)                 │
│    Ruleset::new()                                            │
│      .add_rule(PathBeneath::new("/tmp", WRITE))              │
│      .add_rule(PathBeneath::new("/usr", READ|EXEC))          │
│      .restrict_self()                                        │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 10. Load seccomp filter (POINT OF NO RETURN)                 │
│     ScmpFilterContext::new_filter(Deny)                      │
│       .add_rule(Allow, "read")                               │
```

```
│       .add_rule(Allow, "write")                              │
│       ... add all whitelisted syscalls ...                   │
│       .load()  ← After this, cannot modify filter            │
└──────────────────────────┬──────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│ 11. Execute target binary                                    │
│     execve("/usr/bin/firefox", ["firefox"], env)             │
│     (Process image replaced, sandbox rules remain)           │
└──────────────────────────────────────────────────────────────┘
```

3.3 Data Structures

IsolationConfig

```rust
pub struct IsolationConfig {
    // Filesystem
    pub mounts: Vec<MountConfig>,
    pub filesystem_mode: FilesystemMode,

    // Syscalls
    pub syscall_mode: SyscallMode,
    pub allowed_syscalls: Vec<String>,
    pub syscall_groups: Vec<SyscallGroup>,

    // Network
    pub network_mode: NetworkMode,
    pub allowed_hosts: Vec<String>,

    // Capabilities
    pub allowed_capabilities: Vec<Capability>,

    // Resources
    pub max_memory: Option<u64>,
    pub max_cpu_percent: Option<u32>,
    pub max_processes: Option<u32>,
    pub timeout: Option<Duration>,
}

pub struct MountConfig {
    pub source: PathBuf,
    pub target: PathBuf,
    pub access: AccessMode,  // ReadOnly, ReadWrite
}

pub enum SyscallMode {
    Whitelist,  // Deny all, allow specified
    Blacklist,  // Allow all, deny specified
}

pub enum NetworkMode {
    None,        // No network at all
    Restricted,  // Own namespace, filtered access
    Full,        // Host network namespace
}
```

Profile TOML Format

```
# Example: firefox.toml

[filesystem]
mode = "whitelist"
mounts = [
  { path = "/tmp",              access = "rw" },
  { path = "/usr",              access = "ro" },
  { path = "/lib",              access = "ro" },
  { path = "~/.mozilla",        access = "rw" },
  { path = "~/Downloads",       access = "rw" },
]

[syscalls]
mode = "whitelist"
allow_groups = ["io", "memory", "threading", "network"]
allow = [
  # Graphics/GPU
  "ioctl", "poll", "epoll_wait",
  # X11/Wayland
  "recvmsg", "sendmsg",
]

[network]
mode = "full"  # Browser needs unrestricted internet

[capabilities]
allow = []  # No special capabilities needed

[resources]
max_memory = "2G"
max_cpu = "75%"
max_procs = 100
timeout = "24h"
```

---

4. FEATURES AND DELIVERABLES

4.1 Core Features (MVP - Minimum Viable Product)

Feature 1: Basic Isolation

Description: Execute untrusted binary with namespace isolation **Implementation:**

- Mount, PID, Network, User namespaces
- Basic filesystem whitelisting
- Syscall filtering via seccomp
- Capability dropping

User Story: As a user, I want to run an untrusted binary with minimal permissions so that it cannot access my sensitive files.

Acceptance Criteria:

- Binary executes successfully in isolated environment
- Cannot access files outside whitelisted paths

- Cannot make network connections (network mode = none)
- Cannot execute non-whitelisted syscalls

Feature 2: CLI Interface

Description: User-friendly command-line interface **Commands:**

```
# Quick one-liner
sandbox-rs --no-net --fs /tmp:rw ./binary

# Using profile
sandbox-rs --profile untrusted.toml ./binary

# Paranoid mode (nothing allowed)
sandbox-rs --paranoid ./binary
```

Flags:

- `--profile <file>` : Load configuration from TOML
- `--no-net` : Disable network completely
- `--fs <path:access>` : Whitelist filesystem path (rw/ro)
- `--allow <syscalls>` : Comma-separated syscall list
- `--paranoid` : Minimal permissions (no network, only /tmp rw)
- `--verbose` : Show isolation setup steps

Acceptance Criteria:

- All flags work as documented
- Error messages are clear and actionable
- Help text is comprehensive

Feature 3: Profile System

Description: TOML-based configuration profiles **Implementation:**

- Parser for TOML configuration files
- Profile validation
- Syscall group expansion (io → read, write, open, ...)
- Path expansion (~ → /home/user)

Deliverable: Pre-built profiles for common applications

```
~/.config/sandbox-rs/profiles/
├── browser.toml          # Firefox, Chrome
├── document-viewer.toml  # Evince, Okular
├── media-player.toml     # VLC, MPV
├── office.toml           # LibreOffice
└── untrusted.toml        # Minimal permissions
```

Acceptance Criteria:

- Profiles parse correctly
- Invalid profiles show clear error messages
- Pre-built profiles work for target applications

Feature 4: Resource Limits

Description: Cgroups-based resource control **Implementation:**

- Memory limits (hard kill threshold)
- CPU quota (percentage of cores)
- Process count limits (prevent fork bombs)
- Optional timeout (kill after N seconds)

Configuration Example:

```
[resources]
max_memory = "512M"
max_cpu = "25%"
max_procs = 50
timeout = "5m"
```

Acceptance Criteria:

- Process is killed when exceeding memory limit
- CPU usage stays within quota
- Fork bomb is contained by pid.max
- Process terminates after timeout

4.2 Advanced Features (Phase 2)

Feature 5: Audit Mode

Description: Automatic syscall and filesystem access discovery **Implementation Options:**

1. **strace-based** (simpler, higher overhead)
    - Fork process, run strace, parse output
    - Extract unique syscalls and file paths
    - Generate profile automatically
2. **ptrace-based** (moderate complexity)
    - Use ptrace() to intercept syscalls
    - Build syscall list in real-time
    - Lower overhead than strace
3. **eBPF-based** (complex, lowest overhead)
    - Attach eBPF programs to tracepoints
    - Collect syscall data in kernel space

- Production-grade performance

Usage:

```
# Run in audit mode
sandbox-rs --audit firefox

# Output:
# [AUDIT] Recording syscalls...
# [AUDIT] Run the application normally, then Ctrl+C
#
# ^C
# [AUDIT] Analysis complete:
#   - 87 unique syscalls observed
#   - 12 file paths accessed
#   - Generated profile: firefox.profile.toml
#
# Review and apply:
#   sandbox-rs --profile firefox.profile.toml firefox
```

Generated Profile:

```
# Auto-generated by sandbox-rs audit mode
# Date: 2026-03-15
# Binary: /usr/bin/firefox

[syscalls]
mode = "whitelist"
allow = [
  "read", "write", "open", "close", "mmap",
  # ... all observed syscalls
]

[filesystem]
mounts = [
  { path = "/usr", access = "ro" },
  { path = "~/.mozilla", access = "rw" },
  # ... all accessed paths
]

[network]
# WARNING: Network connections observed to:
#   - 192.168.1.1:443
#   - 8.8.8.8:53
# Consider setting network mode
mode = "full"
```

Acceptance Criteria:

- Captures all syscalls made during execution
- Generates valid TOML profile
- Warns about suspicious behavior (unexpected network, etc.)

Feature 6: Landlock Integration

Description: Modern LSM-based filesystem access control **Requirements:** Kernel 5.13+ **Implementation:**

- Detect kernel support at runtime
- Fallback to mount namespace if unavailable
- More expressive than seccomp for filesystem rules

Advantages:

- Path-based rules (not just syscall-based)
- Can deny directory listing while allowing file read
- Composable with seccomp

Acceptance Criteria:

- Works on supported kernels (5.13+)
- Gracefully degrades on older kernels
- Documentation explains Landlock benefits

4.3 Project Deliverables

4.3.1 Software Deliverables

1. **sandbox-rs binary**
   - Compiled executable for x86_64 Linux
   - Statically linked (no runtime dependencies)
   - Size target: < 5MB
2. **Source code repository**
   - GitHub repository with full history
   - README with installation and usage
   - Comprehensive inline documentation
   - Example profiles directory
3. **Profile library**
   - Minimum 5 pre-built application profiles
   - firefox.toml, chrome.toml, evince.toml, vlc.toml, untrusted.toml
   - Tested and validated
4. **Documentation**
   - User guide (markdown)
   - Architecture documentation (this document)
   - API documentation (cargo doc)
   - Security analysis report

4.3.2 Academic Deliverables

1. **Project report** (French)

- Introduction and problem statement
- State of the art analysis
- Technical architecture
- Implementation details
- Security analysis and testing
- Results and benchmarks
- Conclusion and future work
- 40-60 pages

2. **Presentation** (French)
   - 20-minute oral defense
   - Live demonstration
   - Architecture diagrams
   - Security comparison with existing tools
   - Q&A preparation
3. **Demo video** (English subtitles)
   - 5-7 minutes
   - Show audit mode discovering requirements
   - Show profile-based execution
   - Demonstrate escape attempt failure
   - Compare with Firejail

---

5. IMPLEMENTATION SCHEDULE

5.1 Timeline Overview (10 weeks total)

```
Week 1-2:  Research & Design
Week 3-4:  Core Isolation (MVP)
Week 5-6:  Profile System & CLI
Week 7-8:  Advanced Features & Testing
Week 9:    Security Analysis & Documentation
Week 10:   Final Testing & Report Finalization
```

5.2 Detailed Schedule (Gantt Chart Compatible)

Phase 1: Research & Design (Weeks 1-2)

Duration: 2 weeks (March 10 - March 23, 2026)

Week 1: Research

- **Day 1-2:** Study Linux namespaces documentation

    - Read man pages: namespaces(7), mount_namespaces(7), pid_namespaces(7)
    - Experiment with `unshare` command
    - Deliverable: Technical notes on namespace behavior
- **Day 3-4:** Study seccomp-bpf and capabilities
    - Read seccomp(2), capabilities(7)
    - Analyze existing seccomp profiles (systemd, Chrome)
    - Deliverable: Syscall categorization document
- **Day 5-7:** Analyze existing tools
    - Install and test: Firejail, Bubblewrap, systemd-run
    - Identify vulnerabilities in Firejail (CVE research)
    - Deliverable: Comparative analysis table

Week 2: Design

- **Day 1-3:** Architecture design
    - Component diagram
    - Data structure design
    - Error handling strategy
    - Deliverable: Architecture document (this document)
- **Day 4-5:** Setup development environment
    - Install Rust toolchain
    - Setup project structure (cargo new)
    - Configure dependencies in Cargo.toml
    - Setup CI/CD (GitHub Actions)
    - Deliverable: Working build system
- **Day 6-7:** Prototype namespace creation
    - Write minimal Rust program using nix crate
    - Test unshare() with different namespaces
    - Verify isolation behavior
    - Deliverable: Proof-of-concept code

Milestones:

- ✅ Architecture finalized
- ✅ Development environment ready
- ✅ Namespace prototype working

Phase 2: Core Isolation Implementation (Weeks 3-4)

Duration: 2 weeks (March 24 - April 6, 2026)

Week 3: Namespace & Filesystem Isolation

- **Day 1-2:** Implement namespace creation
  - `src/isolation/namespaces.rs`
  - Functions: setup_mount_ns(), setup_pid_ns(), setup_net_ns()
  - Unit tests for each namespace type
  - Deliverable: Namespace module
- **Day 3-5:** Implement filesystem builder
  - `src/isolation/filesystem.rs`
  - Create tmpfs root
  - Bind mount whitelisted paths
  - Implement pivot_root
  - Handle /proc, /dev special filesystems
  - Deliverable: Filesystem isolation module
- **Day 6-7:** Testing & debugging
  - Test with simple binaries (/bin/ls, /bin/echo)
  - Verify filesystem isolation (cannot see /home)
  - Verify PID isolation (cannot see host processes)
  - Deliverable: Test suite

Week 4: Seccomp & Capabilities

- **Day 1-3:** Implement seccomp filter builder
  - `src/isolation/seccomp.rs`
  - Syscall name → number mapping
  - BPF filter generation using libseccomp
  - Syscall group expansion
  - Deliverable: Seccomp module
- **Day 4-5:** Implement capability dropping
  - `src/isolation/capabilities.rs`
  - Drop all capabilities by default
  - Support selective capability granting
  - Deliverable: Capabilities module
- **Day 6-7:** Integration & testing
  - Combine all isolation primitives
  - Test with complex applications

- Fix bugs and edge cases
- Deliverable: Working MVP

Milestones:

- ✅ Binary can execute in isolated environment
- ✅ Filesystem whitelisting works
- ✅ Seccomp filtering works
- ✅ Basic sandbox functional

Phase 3: Profile System & CLI (Weeks 5-6)

Duration: 2 weeks (April 7 - April 20, 2026)

Week 5: Profile System

- **Day 1-2:** Define profile schema
  - `src/profile/schema.rs`
  - Rust structs with serde annotations
  - TOML parsing and validation
  - Deliverable: Profile data structures
- **Day 3-4:** Implement profile parser
  - `src/profile/parser.rs`
  - Load TOML files
  - Expand syscall groups
  - Path expansion (~, environment variables)
  - Error handling and validation
  - Deliverable: Profile parser module
- **Day 5-7:** Create default profiles
  - Research common application requirements
  - Test applications to determine needs
  - Write profiles:
    - browser.toml (Firefox, Chrome)
    - document-viewer.toml (Evince, Okular)
    - media-player.toml (VLC)
    - office.toml (LibreOffice)
    - untrusted.toml (minimal permissions)
  - Deliverable: Profile library

Week 6: CLI Interface

- **Day 1-3:** Implement CLI with clap
  - `src/cli/mod.rs`
  - Argument parsing
  - Flag validation
  - Help text and examples
  - Deliverable: CLI module
- **Day 4-5:** Integrate CLI with core
  - Connect CLI flags to IsolationConfig
  - Profile loading logic
  - Override system (CLI flags override profile)
  - Deliverable: End-to-end integration
- **Day 6-7:** User testing & refinement
  - Test with real-world scenarios
  - Improve error messages
  - Add verbose logging
  - Write user documentation
  - Deliverable: User guide

Milestones:

- ✅ Profile system functional
- ✅ CLI interface complete
- ✅ Can run applications with profiles
- ✅ User documentation available

Phase 4: Advanced Features (Weeks 7-8)

Duration: 2 weeks (April 21 - May 4, 2026)

Week 7: Resource Limits & Network

- **Day 1-3:** Implement cgroups integration
  - `src/isolation/cgroups.rs`
  - Create cgroup hierarchy
  - Set memory, CPU, PID limits
  - Handle cgroup cleanup
  - Deliverable: Cgroups module

- **Day 4-5:** Implement network modes
  - `src/isolation/network.rs`
  - Network namespace isolation
  - Veth pair creation for restricted mode
  - Bridge configuration
  - Deliverable: Network module
- **Day 6-7:** Testing & optimization
  - Test resource limit enforcement
  - Test network isolation
  - Performance benchmarking
  - Deliverable: Performance report

Week 8: Audit Mode (Optional but Recommended)

- **Day 1-3:** Implement strace-based audit
  - `src/audit/strace.rs`
  - Fork and exec strace
  - Parse strace output
  - Extract syscalls and file paths
  - Deliverable: Basic audit mode
- **Day 4-5:** Implement profile generator
  - `src/audit/generator.rs`
  - Generate TOML from audit data
  - Add warnings for suspicious behavior
  - User review workflow
  - Deliverable: Profile generator
- **Day 6-7:** Polish and testing
  - Test audit mode with various applications
  - Verify generated profiles work
  - Documentation
  - Deliverable: Complete audit feature

Milestones:

- ✅ Resource limits functional
- ✅ Network isolation complete
- ✅ Audit mode working
- ✅ Profile generation automated

Phase 5: Security Analysis & Testing (Week 9)

Duration: 1 week (May 5 - May 11, 2026)

Day 1-2: Escape Testing

- Attempt to bypass sandbox
- Test against known Firejail CVEs
- Document escape attempts and results
- Deliverable: Security test report

Day 3-4: Comparative Analysis

- Benchmark vs Firejail, Bubblewrap
- Performance overhead measurement
- Security comparison matrix
- Deliverable: Comparison table

Day 5: Fuzzing & Stress Testing

- Use AFL or libfuzzer on sandbox code
- Stress test with fork bombs, memory exhaustion
- Fix discovered issues
- Deliverable: Fuzzing report

Day 6-7: Documentation

- Complete API documentation (cargo doc)
- Write security analysis section for report
- Create threat model diagram
- Deliverable: Security documentation

Milestones:

- ✅ Security testing complete
- ✅ No critical vulnerabilities found
- ✅ Performance acceptable
- ✅ Documentation comprehensive

Phase 6: Finalization (Week 10)

Duration: 1 week (May 12 - May 18, 2026)

Day 1-3: Report Writing

- Write academic report (French)
- Include all diagrams and results

- Proofread and format
- Deliverable: Draft report

Day 4-5: Demo Preparation

- Create demo video
- Prepare presentation slides
- Practice oral defense
- Deliverable: Presentation materials

Day 6-7: Final Testing & Submission

- Final integration testing
- Code cleanup and refactoring
- Tag release version (v1.0.0)
- Submit project deliverables
- Deliverable: Final submission

Milestones:

- ✅ Report complete
- ✅ Presentation ready
- ✅ Demo video recorded
- ✅ Project submitted

5.3 Gantt Chart Summary

Phase	Task	Week 1-2	Week 3-4	Week 5-6	Week 7-8	Week 9	Week 10
Phase 1	Research & Design	▇▇▇▇▇▇▇▇					
Phase 2	Core Isolation		▇▇▇▇▇▇▇▇				
Phase 3	Profile & CLI			▇▇▇▇▇▇▇▇			
Phase 4	Advanced Features				▇▇▇▇▇▇▇▇		
Phase 5	Security Testing					▇▇▇▇	
Phase 6	Finalization						▇▇▇▇

---

6. TEAM ORGANIZATION (RACI MATRIX)

6.1 Roles Definition

Since this is an individual final year project, you (Ahmad Swedan) hold all primary roles. However, for the RACI matrix, we'll define roles for different aspects of your work:

R = Responsible (does the work) **A** = Accountable (final approval) **C** = Consulted (provides input) **I** = Informed (kept updated)

6.2 Stakeholders

1. **Ahmad Swedan** - Student/Developer (YOU)
2. **Academic Supervisor** - ESGI Faculty Advisor
3. **Technical Mentor** - External reviewer (if applicable)
4. **Nokia Manager** - Professional context advisor
5. **ESGI Jury** - Final evaluation committee

6.3 RACI Matrix

Task/Deliverable	Ahmad (Student)	Academic Supervisor	Technical Mentor	Nokia Manager	ESGI Jury
Project Planning	R, A	C	C	I	I
Requirements Analysis	R, A	C	I	I	I
Architecture Design	R, A	C	C	I	I
Rust Development	R, A	I	C	I	I
Namespace Implementation	R, A	I	C	I	I
Seccomp Implementation	R, A	I	C	I	I
Profile System	R, A	I	C	I	I
CLI Development	R, A	I	I	I	I
Audit Mode	R, A	I	C	I	I
Security Testing	R, A	C	C	I	I
Performance Testing	R, A	I	C	I	I
Code Review	R, A	I	C	I	I
Documentation (Technical)	R, A	C	I	I	I
Report Writing (French)	R, A	C	I	I	I
Presentation Preparation	R, A	C	I	I	I
Demo Video Creation	R, A	I	I	I	I

Academic Submission	R, A	A	I	I	I
Oral Defense	R, A	I	I	I	A
Final Grading	I	I	I	I	R, A

6.4 Communication Plan

Weekly Progress Meetings

- Ahmad → Academic Supervisor (30 min, weekly)
- Topics: Progress update, blockers, guidance

Bi-weekly Technical Review

- Ahmad → Technical Mentor (if applicable)
- Topics: Architecture decisions, code review

Monthly Nokia Check-in

- Ahmad → Nokia Manager
- Topics: Time management, professional development

Milestone Reviews

- Ahmad → Academic Supervisor
- After Phase 1, 3, 5 completion
- Formal review of deliverables

---

7. RISK MANAGEMENT

7.1 Technical Risks

Risk	Probability	Impact	Mitigation Strategy
Kernel version incompatibility	Medium	High	Test on multiple kernel versions; provide fallback for older kernels (skip Landlock)
Seccomp filter too restrictive	High	Medium	Implement audit mode first; generate profiles automatically
Performance overhead too high	Low	Medium	Benchmark early; optimize critical paths; acceptable overhead < 10%
Namespace escape vulnerability	Low	Critical	Security testing in Phase 5; peer review; compare with known CVEs
Rust learning curve	Medium	Low	Already familiar with Rust; Nokia experience; extensive documentation
Cgroups v2 not available	Low	Medium	Detect at runtime; degrade gracefully to v1 or skip resource limits

7.2 Schedule Risks

Risk	Probability	Impact	Mitigation Strategy
Underestimated complexity	Medium	High	2-week buffer built into schedule; MVP-first approach
Nokia work conflicts	Medium	Medium	Communicate schedule to manager; work evenings/weekends if needed
Scope creep	High	Medium	Strict MVP definition; defer non-critical features to "future work"
Bug discovery in late phase	Medium	High	Continuous testing; automated test suite; early integration testing

7.3 Academic Risks

Risk	Probability	Impact	Mitigation Strategy
Report too technical	Medium	Medium	Balance technical depth with accessibility; include executive summary
Insufficient French documentation	Low	Medium	Write report in French from start; have native speaker review
Demo failure during defense	Low	Critical	Pre-record video backup; test demo multiple times; prepare fallback examples

---

8. SUCCESS CRITERIA

8.1 Functional Criteria

- ✅ Binary executes successfully in isolated environment
- ✅ Filesystem access is limited to whitelisted paths only
- ✅ Network isolation works (no unauthorized connections)
- ✅ Syscall filtering enforced (unauthorized syscalls fail)
- ✅ Resource limits prevent exhaustion attacks
- ✅ Audit mode generates working profiles
- ✅ Pre-built profiles work for target applications

8.2 Performance Criteria

- ✅ Startup overhead < 100ms for simple binaries
- ✅ Runtime overhead < 10% for I/O operations
- ✅ Memory overhead < 50MB for sandbox infrastructure

8.3 Security Criteria

- ✅ No successful sandbox escapes during testing

- ✅ Resistant to known Firejail CVEs
- ✅ Passes fuzzing tests without crashes
- ✅ No memory safety vulnerabilities (Rust guarantees)

8.4 Academic Criteria

- ✅ Report meets ESGI requirements (40-60 pages)
- ✅ Presentation demonstrates understanding
- ✅ Code is well-documented and maintainable
- ✅ Live demo works during defense

---

9. TOOLS AND DEVELOPMENT ENVIRONMENT

9.1 Development Tools

Programming

- Rust 1.75+ (stable channel)
- Cargo (build system and package manager)
- rustfmt (code formatting)
- clippy (linting)

IDE/Editor

- VS Code with rust-analyzer extension
- Or: IntelliJ IDEA with Rust plugin
- Or: Neovim with rust-tools.nvim

Version Control

- Git 2.40+
- GitHub for repository hosting
- GitHub Actions for CI/CD

Testing

- cargo test (unit tests)
- cargo bench (benchmarking)
- AFL / libfuzzer (fuzzing)
- Valgrind (memory analysis, even for Rust)

Documentation

- cargo doc (API documentation)
- mdBook (user guide)
- Mermaid (diagrams)

- LaTeX (academic report)

System Tools

- strace (syscall tracing)
- bpftrace (eBPF tracing)
- perf (performance analysis)
- ltrace (library call tracing)

9.2 Testing Environment

Virtual Machines

- QEMU/KVM for kernel version testing
- Test on Ubuntu 22.04, 24.04
- Test on Arch Linux (rolling release)

Containers

- Docker for reproducible builds
- GitHub Actions runners for CI

Hardware

- Development: Hyprland setup on Arch Linux
- Testing: Multiple kernel versions in VMs
- Target: x86_64 Linux systems

9.3 CI/CD Pipeline

GitHub Actions Workflow:

```
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions-rs/toolchain@v1
        with:
          toolchain: stable
      - run: cargo build --release
      - run: cargo test
      - run: cargo clippy -- -D warnings
      - run: cargo fmt -- --check

  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: cargo audit
```

10. REFERENCES AND RESOURCES

10.1 Academic Papers

1. "Firejail: A Security Sandbox for Linux" - Analysis and CVEs
2. "gVisor: Container Runtime Sandbox" - Google's approach
3. "Landlock LSM: A New Security Framework" - Kernel documentation
4. "seccomp-bpf: A Secure Computing Mode for Linux"

10.2 Technical Documentation

- Linux Kernel Documentation: namespaces(7), seccomp(2), capabilities(7)
- Rust nix crate documentation
- libseccomp documentation
- cgroups v2 documentation

10.3 Open Source Projects

- Firejail: https://github.com/netblue30/firejail
- Bubblewrap: https://github.com/containers/bubblewrap
- youki (OCI runtime in Rust): https://github.com/containers/youki
- Firecracker (AWS microVM): https://github.com/firecracker-microvm/firecracker

10.4 Books

- "The Linux Programming Interface" by Michael Kerrisk
- "Programming Rust" by Jim Blandy and Jason Orendorff
- "The Rust Programming Language" (official book)

---

11. APPENDICES

Appendix A: Syscall Groups Definition

```
pub enum SyscallGroup {
    Io,         // read, write, open, close, lseek, fstat, stat, etc.
    Memory,     // mmap, munmap, mprotect, brk, madvise
    Threading,  // clone, futex, set_tid_address, set_robust_list
    Network,    // socket, connect, send, recv, bind, listen, accept
    Process,    // fork, execve, wait4, kill, getpid, getppid
    Signal,     // rt_sigaction, rt_sigprocmask, rt_sigreturn
    Time,       // clock_gettime, nanosleep, gettimeofday
    Ipc,        // msgget, msgsnd, msgrcv, semget, semop, shmget
}

impl SyscallGroup {
    pub fn expand(&self) -> Vec<&'static str> {
        match self {
```

```
            Self::Io => vec![
                "read", "write", "open", "openat", "close",
                "lseek", "stat", "fstat", "lstat", "newfstatat",
                "readv", "writev", "pread64", "pwrite64",
                "access", "faccessat", "readlink", "readlinkat",
            ],
            Self::Memory => vec![
                "mmap", "munmap", "mprotect", "brk",
                "madvise", "mremap", "msync",
            ],
            // ... etc
        }
    }
}
```

Appendix B: Example Profiles

untrusted.toml (Minimal Permissions)

```
[filesystem]
mode = "whitelist"
mounts = [
  { path = "/tmp", access = "rw" },
  { path = "/usr", access = "ro" },
  { path = "/lib", access = "ro" },
]

[syscalls]
mode = "whitelist"
allow_groups = ["io", "memory"]
allow = ["exit", "exit_group"]

[network]
mode = "none"

[capabilities]
allow = []

[resources]
max_memory = "256M"
max_cpu = "25%"
max_procs = 10
timeout = "5m"
```

Appendix C: Project Repository Structure

```
sandbox-rs/
├── Cargo.toml
├── Cargo.lock
├── README.md
├── LICENSE
├── .github/
│   └── workflows/
│       └── ci.yml
├── src/
│   ├── main.rs
│   ├── lib.rs
│   ├── cli/
```

```
│   │   ├── mod.rs
│   │   └── args.rs
│   ├── profile/
│   │   ├── mod.rs
│   │   ├── schema.rs
│   │   └── parser.rs
│   ├── isolation/
│   │   ├── mod.rs
│   │   ├── namespaces.rs
│   │   ├── filesystem.rs
│   │   ├── seccomp.rs
│   │   ├── capabilities.rs
│   │   ├── cgroups.rs
│   │   └── network.rs
│   ├── audit/
│   │   ├── mod.rs
│   │   ├── strace.rs
│   │   └── generator.rs
│   └── error.rs
├── tests/
│   ├── integration_tests.rs
│   ├── namespace_tests.rs
│   └── seccomp_tests.rs
├── profiles/
│   ├── browser.toml
│   ├── document-viewer.toml
│   ├── media-player.toml
│   ├── office.toml
│   └── untrusted.toml
├── docs/
│   ├── architecture.md
│   ├── user-guide.md
│   └── security-analysis.md
└── examples/
    ├── basic_isolation.rs
    └── custom_profile.rs
```

DOCUMENT METADATA

Project Name: Application Sandboxing System in Rust **Student:** Ahmad Swedan **Institution:** ESGI Paris **Program:** BSc Systems, Networks and Security (2023-2026) **Academic Year:** 2025-2026 **Supervisor:** [To be assigned] **Document Version:** 1.0 **Last Updated:** March 4, 2026 **Total Pages:** 25

SUMMARY FOR GANTT CHART

10-Week Schedule:

1. **Weeks 1-2:** Research & Design
2. **Weeks 3-4:** Core Isolation (Namespaces, Seccomp, Capabilities)
3. **Weeks 5-6:** Profile System & CLI
4. **Weeks 7-8:** Advanced Features (Cgroups, Network, Audit Mode)
5. **Week 9:** Security Analysis & Testing

6. **Week 10:** Finalization & Report

Key Milestones:

- End of Week 2: Architecture finalized
- End of Week 4: MVP functional
- End of Week 6: Full CLI and profiles ready
- End of Week 8: All features complete
- End of Week 9: Security testing done
- End of Week 10: Project submitted

SUMMARY FOR RACI MATRIX

Your Role (Ahmad): Responsible and Accountable for all technical work **Academic Supervisor:** Consulted on academic aspects, Accountable for final submission **Technical Mentor:** Consulted on architecture and security **Nokia Manager:** Informed of progress **ESGI Jury:** Accountable for final evaluation

Communication:

- Weekly meetings with supervisor
- Bi-weekly technical reviews
- Monthly Nokia check-ins
- Milestone reviews after Phases 1, 3, 5

This document provides everything you need for your project report, Gantt chart, and RACI matrix. Let me know if you need any section expanded or modified!